
Conditional Generative Adversarial Networks Reimplementation

Jacob G. Hunter, Mechanical Engineering, PhD Student ¹

Abstract

Conditional Generative Adversarial Networks are an extension of Generative Adversarial Networks where a generative and discriminant model play a min-max game against one another during training. Conditional Generative Adversarial Networks are advantageous in that the input data is described, or conditioned, by a label. Therefore, the generated output can be controlled by a label rather than being randomly generated as is done in traditional Generative Adversarial Networks. In this paper, a Conditional Generative Adversarial Network is built and trained on the MNIST dataset. The network generates grayscale images of the integers zero to nine which appear hand-written. Once sufficiently trained, an experiment is conducted where 100 integers from zero to nine are generated by the network and compared to 100 hand-written integers from the training set. Although a majority of participants correctly distinguished the AI-generated digits from the hand-written digits in a side-by-side comparison, additional results indicate that people are unsure when determining if a digit is generated or hand-written.

1. Introduction

In 2014, Conditional Generative Adversarial Networks (CGANs) were introduced to the machine learning community by [Mirza & Osindero](#). The advantage of CGANs over Generative Adversarial Networks is that they accept input data *with* their associated class labels, allowing the network to produce generated data that is specific to the label. This type of controlled data generation enables researchers and scientists to perform experiments when either the amount of real data is limited or the real data is too noisy and certain factors need to be more tightly controlled. For example, in human-robot interaction, CGANs are used to alter facial expressions of people while controlling for other factors (head pose, lighting, etc.), so robots would only be influenced by the facial expression ([Deng et al., 2019](#)). In another example, a team of researchers sought to identify the presence of COVID-19 from chest CT images ([Loey et al., 2020](#)); however, they needed more data and likely

they needed it quickly due to the time sensitive nature of the pandemic. By conditioning the network with the labels of COVID-19 present and non-present, the researchers generated ten times the number of samples in the original data set. In both the aforementioned examples, CGANs are vital to obtaining specific or enough data to be used in conjunction with other models and techniques. Thus, CGANs are not simply a popular fad for generating unique and creative images, but an actual tool researchers can use to generate data specific to a problem at hand.

In this paper, a Conditional Generative Adversarial Network is trained on the MNIST dataset ([Deng, 2012](#)) to produce numbers which look handwritten between zero and nine. The CGAN in this work is a modified reimplementation of the original CGAN developed to generate digits from the MNIST dataset conditioned on class labels ([Mirza & Osindero, 2014](#)). The purpose of this paper is to demonstrate the power of a CGAN on a simple data set (MNIST) to (i) show that a reimplementation of a CGAN is possible and (ii) demonstrate that AI-generated images of hand-written digits can approach human level handwriting. By solving this problem, simple yet convincing evidence is displayed that CGANs can be used to generate realistic data. While hand-written digits are a simple or toy problem, the generation of specific, additional data has implications for nearly every field of work which requires data.

2. Related Work

2.1. Generated Adversarial Networks

Generative Adversarial Networks (GANs) allow generative models to be trained in a simpler way than they were previously. GANs have an advantage over prior generative models in that they don't rely on computing probabilities, rather they take a different approach and avoid Markov chains altogether ([Mirza & Osindero, 2014](#)); however, several leaders in the field of machine learning note that the high interest in GANs isn't due to this advantage but rather due to the realistic/high quality output they provide ([Goodfellow et al., 2020](#)). The generative capabilities of a GAN are attributed to the minimax game played by two neural networks, a generator and a discriminator.

The generator is often compared to an art forger who tries

to fool the discriminator, the art expert, with its forgeries; however, the “art forger” doesn’t have access to original art while the “expert” does (Creswell et al., 2018). Therefore, the only way for the generator to improve its forgeries, is by taking feedback from the discriminator. The discriminator also takes feedback for how well it distinguished between forgeries and real pieces. In this fashion, the generator and discriminator are trained simultaneously (Goodfellow et al., 2020; Mirza & Osindero, 2014).

Although not the only application, a popular use of GANs is to generate images. For example, if a GAN is trained to generate images of cars, the training set will include tens or even hundreds of thousands of vehicle images. These vehicles will likely include sedans, minivans, trucks, SUVs, etc. Thus, once the model is trained and an image of a car is desired, any of the aforementioned vehicle types may be generated. There is no way to control which kind of car will be produced by the model; nevertheless, some sort of car will be displayed.

In contrast, CGANs allow for a label to be attached to the data being input into a generative model. The academic community calls this “conditioning” the data. In fact, in the original GAN paper in 2014, Goodfellow et al. the future work describes a conditional generative model, which is really the CGAN. Later that year, Mirza & Osindero publish the paper on CGANs. Now, returning to the example of generating vehicle images, each image of a car given to the GAN can have their respective vehicle type label attached (e.g., sedan, minivan, truck, SUV). Therefore, once the model is trained, it is easy to generate a specific kind of vehicle by feeding the generator both the random noise as well as the label of the desired vehicle. Therefore, CGANs do require labelling but provide an advantage over GANs for certain contexts.

2.2. MNIST

The MNIST database is “a collection of handwritten digit images used extensively in optical character recognition and machine learning research” (Deng, 2012). There are 60,000 training samples and 10,000 test samples. The MNIST data set was originally introduced 25 years ago and was created by taking handwritten digits from a couple of NIST’s special databases (LeCun et al., 1998). The MNIST data set is simple to use, takes nearly no storage, and is used frequently in machine learning to quickly test new algorithms, across different programming languages; however, there have been calls from within the community to move away from MNIST and subsequent data sets released (Xiao et al., 2017; Cohen et al., 2017). For example, Xiao et al. created a plug-and-play substitute for MNIST, titled Fashion-MNIST, which is filled with grayscale images of fashion accessories (size 28x28). In addition to Fashion-MNIST, another data

set known as EMNIST was created by Cohen et al.; the EMNIST data set is built in a similar way to how the original MNIST data set was constructed, but includes letters and numbers. Both of these new data sets provide a more challenging classification task than classifying data in MNIST and are currently serving as functional benchmarks. Nevertheless, the simplicity and ease of using MNIST appears to remain in force and is, understandably, a great starting place for beginners new to machine learning.

While the MNIST data set is frequently used with classification algorithms, it is also used with GANs. In addition to using the data sets CIFAR-10 and SVHN, researchers at OpenAI used the MNIST data set to validate their new GAN architectures and training techniques (mini-batch discrimination) (Salimans et al., 2016). Other researchers noted the various GAN architectures that exist and evaluated many of the popular ones against each other on the MNIST data set (Cheng et al., 2020). Therefore, despite its simplicity, the MNIST data set is an effective tool to use for not only classification, but generative adversarial networks.

For the purposes of this work, the CGAN is trained on the 60,000 training samples of the MNIST data set.

3. Methodology

3.1. Labels

The class labels zero through nine are applied to the MNIST images. A one-hot encoding of the numerical labels is used to ensure the machine learning models view the labels as independent during training. Although one-hot encoding requires converting labels to vectors by storing a dictionary (Seeger, 2018), only requiring ten labels for the current implementation allowed memory resources to not be adversely impacted.

3.2. Neural Network Models

The generator and discriminator are each comprised of a neural network. After several attempts to create the generator and discriminator neural networks based on the paper and/or from scratch, an existing generator and discriminator implementation were used (Schwarz, 2022). However, the generator and discriminator code were not copied, but the neural network layers were noted and then implemented by the author.

3.2.1. GENERATOR

The generator neural network architecture is defined by five linear layers, four LeakyReLU activation layers with a negative slope of 0.2, three BatchNorm1d layers, and a final non-linear activation layer Tanh. Before the first linear layer, noise sampled from a uniform distribution (size 100)

Table 1. Generator neural network architecture. Architecture borrowed from (Schwarz, 2022)

	Layer Type	Input	Output	Neg. Slope
1	Linear	100 + 10	128	-
2	LeakyReLU	-	-	0.2
3	Linear	128	256	-
4	BatchNorm1d	256	256	-
5	LeakyReLU	-	-	0.2
6	Linear	256	512	-
7	BatchNorm1d	512	512	-
8	LeakyReLU	-	-	0.2
9	Linear	512	1024	-
10	BatchNorm1d	1024	1024	-
11	LeakyReLU	-	-	0.2
12	Linear	1024	64x64	-
13	Tanh	-	1 or 0	-

Table 2. Discriminator neural network architecture. Architecture borrowed from (Schwarz, 2022)

	Layer Type	Input	Output	Neg. Slope
1	Linear	(64*64) + 10	1024	-
2	LeakyReLU	-	-	0.2
3	Linear	1024	512	-
4	Dropout	-	-	-
5	LeakyReLU	-	-	0.2
6	Linear	512	256	-
7	Dropout	-	-	-
8	LeakyReLU	-	-	0.2
9	Linear	256	128	-
10	Linear	128	1	-
11	Sigmoid	-	1 or 0	-

is concatenated with the ten labels (size 10); thus, the input to the neural network has a size of 110. For the complete architecture ordering, see Table 1.

3.2.2. DISCRIMINATOR

The discriminator neural network architecture is defined by five linear layers, three LeakyReLU activation layers, two dropout layers, and one final, non-linear activation sigmoid layer. Before the first linear layer, the MNIST image dimensions (generated or real) are flattened (size 4096) then concatenated with the class labels (size 10). Note that the input image dimensions are 64x64 because Schwarz recommended resizing the MNIST images from the 28x28 size upon import.

3.3. Loss function

As described in (Mirza & Osindero, 2014), the following loss function is optimized in the training of the CGAN:

$$\min_G \max_D V(D, G) = E_{x \sim p_{\text{data}}(x)} [\log D(x|y)] + E_{z \sim p_z(z)} [1 - \log D(G(z|y))]$$

The above equation is a form of binary cross-entropy loss. The expectation of the log-likelihood of the discriminator, the first term, is given in equation 1 below:

$$E_{x \sim p_{\text{data}}(x)} [\log D(x|y)] \quad (1)$$

This term takes the inputs of the real MNIST data and the labels; therefore, the output should be minimized because the true data should match the true labels.

The second term in the loss function is given in equation 2 below:

$$E_{z \sim p_z(z)} [1 - \log D(G(z|y))] \quad (2)$$

In this term, the generator creates a fake MNIST image and then the discriminator compares this fake image to the label. The generator loss wants to be as small as possible, meaning its outputs match the true labels as much as possible. Then the discriminator wants to minimize its loss by deciding which fake images are not real.

By each trying to minimize their own loss, the two models are trained simultaneously.

3.4. Model training

As suggested in the paper (Mirza & Osindero, 2014), the model was trained using stochastic gradient descent rather than the Adam algorithm as done in other implementations. A batch size of 100 was used. Additionally, both the generator and discriminator optimizers each used a learning rate of 0.1; however, a weight decay factor of 0.00005 was implemented instead of the recommended in the paper. Lastly, momentum was set to the value 0.5, but not increased up to 0.7 as Mirza & Osindero did. For simplicity, 200 epochs were used as the stopping point for model training rather than enacting an automated stopping approach. This was determined by referencing an existing implementation that used 200 epochs.

The following generated images show the network's training progression at epoch 12 (Figure 1), epoch 51 (Figure 2), and epoch 200 (Figure 3).

To ensure that the generator did become conditioned on the correct labels, the labels zero to nine were placed in order a total of 10 times and fed as an input to the generator along with the sampled noise. The output was as expected with the generated "hand-written" labels in order from zero to

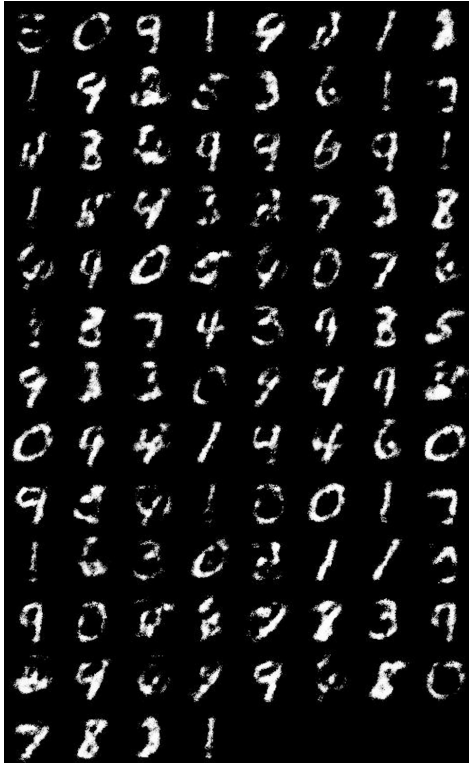


Figure 1. Conditional generative adversarial network training output at epoch 12



Figure 2. Conditional generative adversarial network training output at epoch 51

nine, repeated 10 times, as seen in Figure 4. Therefore, the generator was trained correctly and, accordingly, the discriminator correctly performed its proper function.

4. Experiment

Perhaps the best baseline to compare generated, “handwritten” digits to is the training data itself. Previous work has shown that MNIST digits can be generated, using a GAN, that are indistinguishable from real handwritten data (Salimans et al., 2016). Therefore, in like manner, a comparison experiment between generated handwritten digits (generated by the CGAN described in this paper) and real handwritten digits (from the MNIST training set) was deployed on Instagram.

4.1. Experiment design

A three-step experiment was conducted on an Instagram story where followers could respond whether they thought handwritten images were AI-generated or actually handwritten.

The first step of the experiment showed followers 100 generated digits and asked them to choose if the digits were (a) Handwritten or (b) AI Generated, see Figure 5. The digits

in the first step are AI-generated from the CGAN.

After making their choice, participants proceeded to the second step of the experiment where, on a new page, they were shown 100 handwritten digits. On this page, they were again asked to choose if the digits were (a) Handwritten or (b) AI Generated, see Figure 6. The digits in the second step are handwritten and taken from the MNIST training set. Note that the AI-generated digit labels match exactly the handwritten digits labels (same digits and same order).

After completing the second step of the experiment, participants are led to a final Instagram story page to complete the third step of the experiment. Two images, AI-generated and handwritten, are shown side-by-side. This can be seen in Figure 7.

4.2. Experiment Results

4.2.1. STEP ONE

The first step of the experiment which consisted of AI-generated images had 17 respondents. Thirteen respondents (76%) chose correctly that the digits were AI-generated. Four respondents (20%) chose incorrectly that the digits were handwritten.

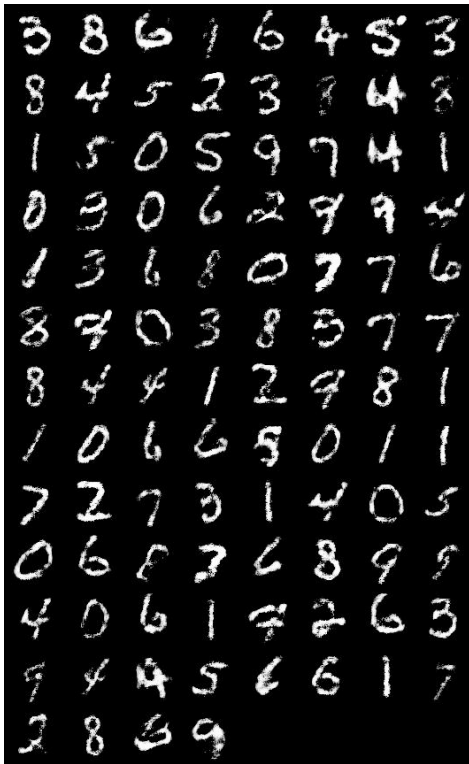


Figure 3. Conditional generative adversarial network output at the conclusion of training (epoch 200)



Figure 4. Fully-trained generator output of labels zero to nine in order, repeated 10 times

4.2.2. STEP TWO

The second step of the experiment which consisted of handwritten images had 17 respondents. Eleven respondents (65%) chose correctly that the digits were handwritten. Six respondents (35%) chose incorrectly that the digits were AI-generated.

4.2.3. STEP THREE

The third step of the experiment consisted of both AI-generated digits and handwritten digits displayed side-by-side. This step had 19 respondents. Twelve respondents (63%) chose correctly that the digits on the left were AI-generated. Seven respondents (37%) chose incorrectly that the handwritten digits on the right were AI-generated.

4.3. Experiment Discussion

Interestingly, more people chose that an AI had generated the actual handwritten digits during step two than the number of people who chose that the digits were handwritten in step one. Although it is a small sample size ($n=17$), this preliminary result indicates that some people believe AI is capable of generating increasingly realistic data.

Since a majority of respondents (76%) chose correctly dur-

ing the first step that the digits were generated artificially, this suggests that my generator and discriminator algorithms can be further improved. In Figures 4 and 5, one can see that some of the digits aren't quite clear.

Lastly, step three of the experiment provides interesting results in that seven people thought the AI-generated digits were actually handwritten. This may have occurred due to having two *smaller* images on the same size screen, so the resolution of the digits looked better for the AI-generated images.

5. Conclusion

This paper demonstrates a reimplementation of a Conditional Generative Adversarial Network to generate “handwritten” images. The CGAN consisted of a generator and discriminator which were trained simultaneously on the training set data from the MNIST database. An experiment was carried out on Instagram where followers were asked to make a guess whether one hundred digits were AI-generated or handwritten. This was asked of participants twice, once with AI-generated digits and once with handwritten digits. Finally, followers were asked to decide which one hundred digits were AI-generated in a side-by-side comparison. Overall, the majority of respondents were

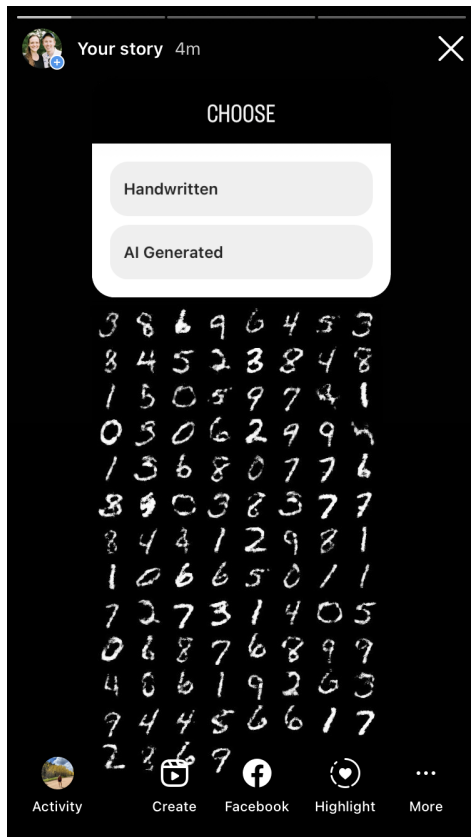


Figure 5. First step of the experiment; AI-generated images from the trained CGAN are shown to Instagram followers; an Instagram story poll is deployed to collect data

able to decipher which ones were AI-generated from the CGAN; however, some of the data points toward respondents confusing whether real data is actually AI-generated or not.

Lastly, there were several limitations to this work. First, the CGAN was trained for two hundred epochs and this was chosen arbitrarily. In the future, a stopping criterion which takes into account the loss may prove useful in deciding how long to train. Second, the neural networks of the generator and discriminator did a decent job; however, even after training, some of the digits still didn't look quite correct, although nearly all were readable. Third, the experiment was subject to several biases. Ordering effects were not counterbalanced. The AI-generated images were always shown first to participants while the handwritten images were shown second. Fourth, not all Instagram followers responded to the experiment despite seeing it on their feed. These respondents may have been unsure or not confident in their answer, and thus decided to skip the survey; therefore, valuable data points were lost. Fifth, one hundred MNIST digits were shown in a block rather than in a realistic setting. It would be interesting to take the AI-generated digits and

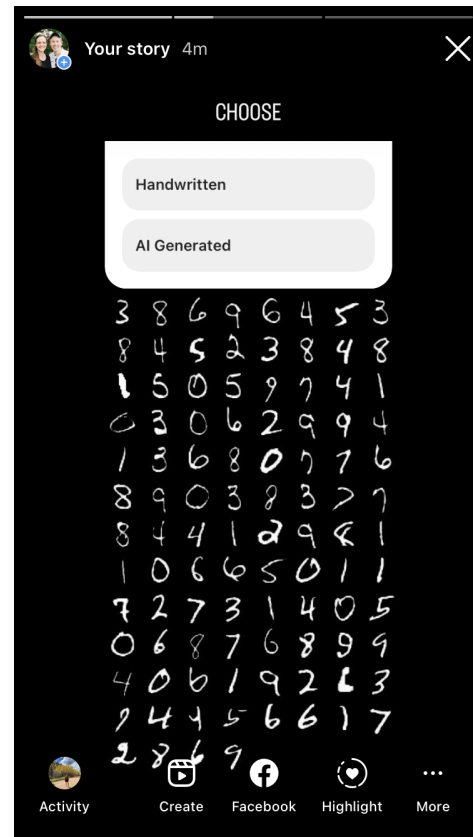


Figure 6. Second step of the experiment; handwritten images from the MNIST training set are shown to Instagram followers; an Instagram story poll is deployed to collect data

put them on envelopes, for example, and see if people think they were actually written by someone. The experiment in this paper surveyed people completely digitally on a small screen, which may have led to incorrect guesses.

Acknowledgements

I would like to acknowledge Ruqi Bai and Wei Chen for their significant amount of help on this project. Ruqi went above and beyond helping me to understand CGANs and guiding in me in architecture development. Although I ultimately failed at developing my own neural networks for the generator and discriminator (and used “off-the-shelf” networks), I learned a lot about developing one's own neural network. Second, I would like to acknowledge Dr. Guo who pointed me to use the binary cross-entropy loss and helped me understand how to apply it in PyTorch. Third, I would also like to acknowledge that I used a few lines of code from the [Schwarz](#) implementation in order to get my training loop to run correctly. Specifically, I was having an issue where I couldn't generate anything with my training loop. The specific PyTorch code that it helped me with was the

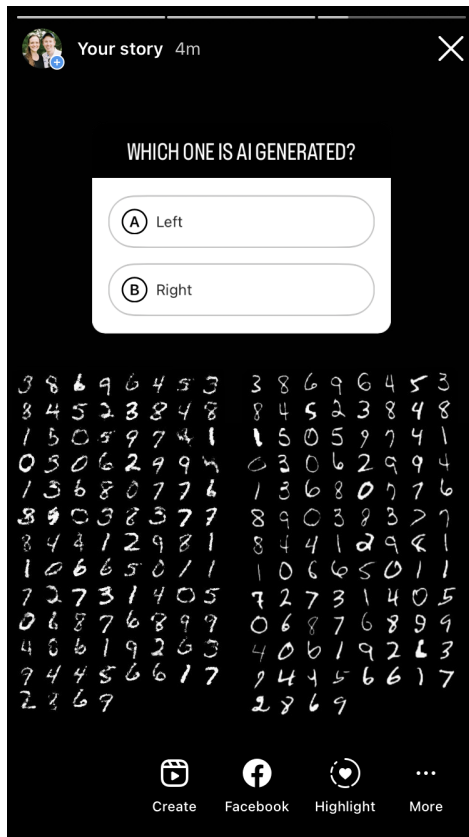


Figure 7. Third step of the experiment; AI-generated images and handwritten images shown side by side to Instagram followers; an Instagram story quiz is deployed to collect data

“.detach()” command, so I could use my fake image even after backpropagating the gradients. Additionally, it helped me know where in the training loop to set my gradients to zero. Lastly, I would like to acknowledge the PyTorch documentation which served as a guide to me while learning how to create a machine learning model with neural networks.

References

- Cheng, K., Tahir, R., Eric, L. K., and Li, M. An analysis of generative adversarial networks and variants for image synthesis on mnist dataset. *Multimedia Tools and Applications*, 79:13725–13752, 2020.
- Cohen, G., Afshar, S., Tapson, J., and Van Schaik, A. Emnist: Extending mnist to handwritten letters. In *2017 international joint conference on neural networks (IJCNN)*, pp. 2921–2926. IEEE, 2017.
- Creswell, A., White, T., Dumoulin, V., Arulkumaran, K., Sengupta, B., and Bharath, A. A. Generative adversarial networks: An overview. *IEEE signal processing magazine*, 35(1):53–65, 2018.
- Deng, J., Pang, G., Zhang, Z., Pang, Z., Yang, H., and Yang, G. cgan based facial expression recognition for human-robot interaction. *IEEE Access*, 7:9848–9859, 2019.
- Deng, L. The mnist database of handwritten digit images for machine learning research [best of the web]. *IEEE signal processing magazine*, 29(6):141–142, 2012.
- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., and Bengio, Y. Generative adversarial nets. 2014.
- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., and Bengio, Y. Generative adversarial networks. *Communications of the ACM*, 63(11):139–144, 2020.
- LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- Loey, M., Manogaran, G., and Khalifa, N. E. M. A deep transfer learning model with classical data augmentation and cgan to detect covid-19 from chest ct radiography digital images. *Neural Computing and Applications*, pp. 1–13, 2020.
- Mirza, M. and Osindero, S. Conditional generative adversarial nets. *arXiv preprint arXiv:1411.1784*, 2014.
- Salimans, T., Goodfellow, I., Zaremba, W., Cheung, V., Radford, A., and Chen, X. Improved techniques for training gans. *Advances in neural information processing systems*, 29, 2016.
- Schwarz, S. How to build a cgan for generating mnist digits in pytorch. *Medium*, 2022.
- Seeger, C. An investigation of categorical variable encoding techniques in machine learning: binary versus one-hot and feature hashing, 2018.
- Xiao, H., Rasul, K., and Vollgraf, R. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017.